



l'école d'ingénierie
informatique

CERTIFICATION DÉVELOPPEUR EN INTELLIGENCE ARTIFICIELLE ET DATA SCIENCE RNCP 36581

BLOC DE COMPÉTENCES 1 : Créer un modèle de données d'une solution I.A en utilisant des méthodes de Data sciences

Développement et déploiement d'une application dans le respect du cahier des charges Client - Création d'un backend métier permettant le nettoyage et la visualisation des données.

SOMMAIRE

Introduction

Modèle de BDD, UML et script

- I. Modèle de la BDD
- II. UML (MCD, MLD, MPD)

ETL : gestion et traitement des jeux de données

- I. Dataset d'entrée
- II. Benchmark
- III. Processus de nettoyage
- IV. Traitement et réorganisation

API : Obtention et manipulation des données

- I. Mise en place de l'API
- II. Doc de l'API

PowerBI : solution de visualisation

- I. Benchmark
- II. Explications des visualisations

Conclusion

Annexe

Introduction

Afin de répondre aux exigences de la Mise en Situation Professionnelle Reconstituée TPRES01 et de poser les bases d'une solution basée sur l'intelligence artificielle, nous avons choisi d'analyser les données liées à l'épidémie du VIH (Virus de l'Immunodéficience Humaine). Ce virus s'attaque au système immunitaire et réduit les défenses naturelles de l'organisme, rendant les individus plus vulnérables aux infections et maladies opportunistes.

L'objectif de ce projet est d'exploiter ces données afin de mieux comprendre leur structure, d'en extraire des informations pertinentes et de proposer une solution facilitant leur analyse et leur visualisation. Pour cela, nous avons structuré notre travail en plusieurs étapes clés :

- **Modélisation de la base de données (BDD) et conception UML** : Cette phase vise à définir l'architecture des données, en modélisant les relations entre les différentes entités afin d'assurer une organisation cohérente et optimisée pour les traitements à venir.
- **Mise en place d'un processus ETL (Extract, Transform, Load)** : Cette étape consiste à récupérer les jeux de données, les nettoyer et les transformer en formats exploitables, avant de les charger dans une base de données. Un travail de normalisation a été mené pour garantir la qualité des informations utilisées.
- **Développement d'une API pour l'obtention et la manipulation des données** : Afin de faciliter l'accès aux données et d'assurer leur exploitation par différents systèmes, nous avons mis en place une API permettant d'interroger la base de données de manière efficace. Une documentation détaillée accompagne cette API pour en garantir une utilisation optimale.
- **Visualisation des données avec Power BI** : Pour rendre les résultats plus accessibles et compréhensibles, nous avons utilisé Power BI afin de créer des dashboards interactifs. Un benchmark des différentes solutions de visualisation a été effectué pour justifier ce choix, et les visualisations mises en place permettent d'explorer les tendances clés et d'identifier des informations stratégiques issues des données.

Ce rapport détaille chacune de ces étapes en mettant en avant les choix méthodologiques, les outils utilisés et les résultats obtenus.

Modèle de BDD, UML et script

I. Modèle de la BDD

Après analyse et découpage préalable des jeux de données en notre possession, nous avons élaboré un modèle théorique de notre base de données. Dans un premier temps nous avons cherché à savoir quels seraient les éléments les plus importants et comment les organiser entre eux afin de garantir une cohérence de la donnée. Nous avons donc isolé dans une première strate les données concernant les pays, les statistiques et les traitements, qui sont ressortis à nos yeux comme les valeurs les plus importantes. Dans un second temps nous avons réunis dans une seconde strate les types de traitements, la population de personnes atteintes du VIH, les éléments liés à la mortalité, les données sur la transmission mère-enfant, le type de statistique et enfin les unités de ces mêmes statistiques.

```
-- =====
-- Script : create_tables.sql
-- Description :
-- Ce script crée toutes les tables nécessaires dans la base de données :
-- - Tables de référence (pays, unités, types)
-- - Tables de données (population_hiv, mortalité, transmission, traitements)
-- - Tables de statistiques
-- Il définit également toutes les contraintes et relations entre les tables.
-- =====

-- Création des tables de référence
CREATE TABLE IF NOT EXISTS pays (
    id_pays SERIAL PRIMARY KEY,
    nom_pays VARCHAR(100) NOT NULL,
    region VARCHAR(100),
    sous_region VARCHAR(100)
);

CREATE TABLE IF NOT EXISTS unite (
    id_unite SERIAL PRIMARY KEY,
    nom_unite VARCHAR(50) NOT NULL
);

CREATE TABLE IF NOT EXISTS type_statistique (
    id_type_statistique SERIAL PRIMARY KEY,
    nom_type_statistique VARCHAR(100) NOT NULL
);

CREATE TABLE IF NOT EXISTS type_traitement (
    id_type_traitement SERIAL PRIMARY KEY,
    nom_type_traitement VARCHAR(100) NOT NULL
);
```

II. UML (MCD, MLD, MPD)

Modèle Conceptuel de Données (MCD) (schéma en annexe)

Entités et leurs attributs :

Tables de référence :

pays_clean

- id_pays (INT, Clé primaire)
- pays (VARCHAR(100))
- region_who (VARCHAR(100))

unite_clean

- id_unite (INT, Clé primaire)
- libelle (VARCHAR(50))

type_statistique_clean

- id_type_statistique (INT, Clé primaire)
- libelle (VARCHAR(100))

type_traitement_clean

- id_type_traitement (INT, Clé primaire)
- libelle (VARCHAR(100))

Tables principales :

table_population_hiv

- id_pays (INT, Clé étrangère → pays_clean)
- annee (INT)
- population_min (INT)
- population_median (INT)
- population_max (INT)
- id_unite (INT, Clé étrangère → unite_clean)
- Index sur (annee, id_pays)

table_mortalite

- id_pays (INT, Clé étrangère → pays_clean)
- annee (INT)

- mortalite_min (INT)
- mortalite_median (INT)
- mortalite_max (INT)
- id_unite (INT, Clé étrangère → unite_clean)
- Index sur (annee, id_pays)

table_transmission_mere_enfant

- id_transmission (INT, Clé primaire)
- id_pays (INT, Clé étrangère → pays_clean)
- besoin_arv_min (INT)
- besoin_arv_median (INT)
- besoin_arv_max (INT)
- pourcentage_recu_min (FLOAT)
- pourcentage_recu_median (FLOAT)
- pourcentage_recu_max (FLOAT)
- Index sur (id_pays)

table_traitement

- id_traitement (INT, Clé primaire)
- id_pays (INT, Clé étrangère → pays_clean)
- annee (INT)
- id_type_traitement (INT, Clé étrangère → type_traitement_clean)
- couverture (FLOAT)
- Index sur (annee, id_pays)

table_statistique

- id_statistique (INT, Clé primaire)
- id_pays (INT, Clé étrangère → pays_clean)
- annee (INT)
- id_type_statistique (INT, Clé étrangère → type_statistique_clean)
- valeur (FLOAT)
- id_unite (INT, Clé étrangère → unite_clean)
- Index sur (annee, id_pays)

Relations :

- **pays** est en relation avec **traitement**, **statistique**, **population_hiv**, **mortalite**, et **transmission_mere_enfant** (1,N)
- **unite** est en relation avec **statistique** (1,N)

- **type_statistique** est en relation avec **statistique** (1,N)
- **type_traitement** est en relation avec **traitement** (1,N)

Modèle logique de données (MLD)

pays_clean (id_pays PK, pays, region_who)

unite_clean (id_unite PK, libelle)

type_statistique_clean (id_type_statistique PK, libelle)

type_traitement_clean (id_type_traitement PK, libelle)

table_population_hiv (id_pays FK → pays_clean, annee, population_min, population_median, population_max, id_unite FK → unite_clean)

table_mortalite (id_pays FK → pays_clean, annee, mortalite_min, mortalite_median, mortalite_max, id_unite FK → unite_clean)

table_transmission_mere_enfant (id_transmission PK, id_pays FK → pays_clean, besoin_arv_min, besoin_arv_median, besoin_arv_max, pourcentage_recu_min, pourcentage_recu_median, pourcentage_recu_max)

table_traitement (id_traitement PK, id_pays FK → pays_clean, annee, id_type_traitement FK → type_traitement_clean, couverture)

table_statistique (id_statistique PK, id_pays FK → pays_clean, annee, id_type_statistique FK → type_statistique_clean, valeur, id_unite FK → unite_clean)

Modèle Physique de Données (MPD - SQL)(voir annexe)

```
CREATE TABLE pays_clean (  
    id_pays INT PRIMARY KEY AUTO_INCREMENT,  
    pays VARCHAR(100),  
    region_who VARCHAR(100)  
);
```

```
CREATE TABLE unite_clean (  
    id_unite INT PRIMARY KEY AUTO_INCREMENT,  
    libelle VARCHAR(50)  
);
```

```
CREATE TABLE type_statistique_clean (  
    id_type_statistique INT PRIMARY KEY AUTO_INCREMENT,  
    libelle VARCHAR(100)  
);
```

```
CREATE TABLE type_traitement_clean (  
    id_type_traitement INT PRIMARY KEY AUTO_INCREMENT,  
    libelle VARCHAR(100)  
);
```

```
CREATE TABLE table_population_hiv (  
    id_pays INT,  
    annee INT,  
    population_min INT,  
    population_median INT,  
    population_max INT,  
    id_unite INT,  
    PRIMARY KEY (id_pays, annee),  
    FOREIGN KEY (id_pays) REFERENCES pays_clean(id_pays),  
    FOREIGN KEY (id_unite) REFERENCES unite_clean(id_unite),  
    INDEX idx_population_hiv (annee, id_pays)  
);
```

```
CREATE TABLE table_mortalite (  
    id_pays INT,  
    annee INT,  
    mortalite_min INT,  
    mortalite_median INT,  
    mortalite_max INT,  
    id_unite INT,  
    PRIMARY KEY (id_pays, annee),  
    FOREIGN KEY (id_pays) REFERENCES pays_clean(id_pays),  
    FOREIGN KEY (id_unite) REFERENCES unite_clean(id_unite),  
    INDEX idx_mortalite (annee, id_pays)  
);
```

```
CREATE TABLE table_transmission_mere_enfant (  
    id_transmission INT PRIMARY KEY AUTO_INCREMENT,  
    id_pays INT,  
    besoin_arv_min INT,  
    besoin_arv_median INT,  
    besoin_arv_max INT,  
    pourcentage_recu_min FLOAT,  
    pourcentage_recu_median FLOAT,  
    pourcentage_recu_max FLOAT,  
    FOREIGN KEY (id_pays) REFERENCES pays_clean(id_pays),  
    INDEX idx_transmission_mere_enfant (id_pays)  
);
```

```
CREATE TABLE table_traitement (  
    id_traitement INT PRIMARY KEY AUTO_INCREMENT,  
    id_pays INT,  
    annee INT,  
    id_type_traitement INT,  
    couverture FLOAT,  
    FOREIGN KEY (id_pays) REFERENCES pays_clean(id_pays),  
    FOREIGN KEY (id_type_traitement) REFERENCES type_traitement_clean(id_type_traitement),  
    INDEX idx_traitement (annee, id_pays)  
);
```

```
CREATE TABLE table_statistique (  

```

```
id_statistique INT PRIMARY KEY AUTO_INCREMENT,  
id_pays INT,  
annee INT,  
id_type_statistique INT,  
valeur FLOAT,  
id_unite INT,  
FOREIGN KEY (id_pays) REFERENCES pays_clean(id_pays),  
FOREIGN KEY (id_type_statistique) REFERENCES type_statistique_clean(id_type_statistique),  
FOREIGN KEY (id_unite) REFERENCES unite_clean(id_unite),  
INDEX idx_statistique (annee, id_pays)  
);
```

ETL : Gestion et traitement des jeux de données

I. Dataset d'entrée

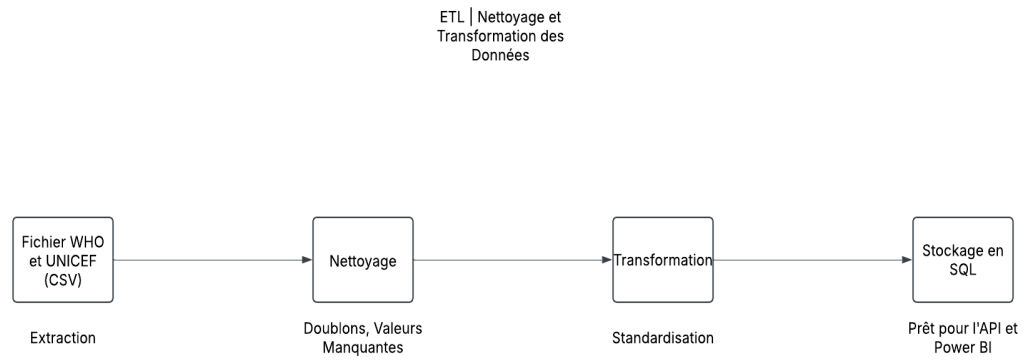
Nous avons utilisé un set de données trouvé sur [kaggle](#). Il comporte les informations relatives au virus du VIH, selon deux sources fiables, l'OMS (WHO en Anglais) et l'Unesco réparties en six fichiers CSV.

II. Benchmark

Afin de mettre en place un système de traitement nous avons évalué les diverses possibilités qui s'offrent à nous. Nous avons comparé Pandas, Talend, Apache HOP ainsi qu'Airflow, selon les 5 critères suivants : la performance de traitement des flux de données, la simplicité d'utilisation, la scalabilité (manière dont le système s'adapte aux modulations des volumes exploités), la compatibilité du système avec divers formats de documents, et enfin le coût que représente l'utilisation du logiciel. (voir annexe)

Nous avons jeté notre dévolu sur l'utilisation de Pandas, qui nous permet de travailler avec la même technologie que notre API, mais surtout qui correspond plus à nos besoins, afin de produire un résultat cohérent avec l'ensemble du projet. Le fait qu'il soit Open Source a été un facteur décisionnel étant donné que la communauté Python est très active et permet de trouver facilement des informations en cas de besoin.

III. Processus de nettoyage



```
# ----- ● EXTRACTION (Extract) -----
Tabnine | Edit | Test | Explain | Document
def extract_data():
    """
    EXTRACTION : Lecture des fichiers sources
    Returns:
        list: Liste des DataFrames extraits ou None en cas d'erreur
    """
    try:
        logging.info("🔵 Début de l'extraction...")

        files = [
            "../SourceData/art_coverage_by_country_clean.csv",
            "../SourceData/art_pediatric_coverage_by_country_clean.csv",
            "../SourceData/no_of_cases_adults_15_to_49_by_country_clean.csv",
            "../SourceData/no_of_deaths_by_country_clean.csv",
            "../SourceData/no_of_people_living_with_hiv_by_country_clean.csv",
            "../SourceData/prevention_of_mother_to_child_transmission_by_country_clean.csv"
        ]

        dataframes = []
        for file_path in files:
            path = Path(file_path)
            if not path.exists():
                raise FileNotFoundError(f"Fichier introuvable : {file_path}")

            df = pd.read_csv(file_path)
            if df.empty:
                logging.warning(f"⚠ Le fichier {file_path} est vide")
                continue
```

```

# Vérification des colonnes nécessaires
country_col = next((col for col in df.columns if "Country" in col), None)
region_col = next((col for col in df.columns if "WHO Region" in col), None)

.....

if country_col:
    df.rename(columns={country_col: "pays"}, inplace=True)
    if region_col:
        df.rename(columns={region_col: "region_who"}, inplace=True)
        df = df[["pays", "region_who"]].drop_duplicates()
    else:
        df = df[["pays"]].drop_duplicates()
    dataframes.append(df)
else:
    logging.warning(f"⚠️ Aucune colonne 'Country' trouvée dans {file_path}")
    continue

...

if not dataframes:
    raise ValueError("Aucune donnée valide extraite des fichiers sources")

.....

logging.info("✅ Extraction réussie")
return dataframes

```

Nous avons entamé le processus de nettoyage en prenant les différentes tables en compte et en les normalisant, pour cela nous avons renommé les colonnes comportant le pays dans chaque fichier, ainsi que supprimer les espaces. Chaque colonne a vu son nom être transformé en minuscules. Certains fichiers comportaient aussi une mention de la région géographique, qui a subi le même traitement. Chacune de ces colonnes a aussi été scrutée par la fonction `drop_duplicates()` afin d'écarter les doublons. Nous avons organisé les pays en un seul fichier qui sert de référence à chaque autre fichiers inclus dans le set de base, ce qui nous permet par la suite de traiter chaque fichier afin qu'il corresponde à ce dernier et nous a permis d'éliminer les données non conformes, qui n'avaient pas d'informations suffisantes ou qui n'avaient pas les bons ID de pays par exemple.

Les données concernant les unités ont été transformées en table afin d'offrir un indice et une clé spéciale les concernant dans notre base de données. Un traitement similaire a été appliqué aux fichiers concernant les types de statistique (taux de HIV, taux de mortalité, et nombre de cas) ainsi qu'aux types de traitements (adultes ou enfants).

IV. Traitement et réorganisation

Nous avons commencé notre travail de traitement par la réunion des données en fonction de la table des pays qui a été mise en place grâce au nom de pays, son id, et la région de l'OMS à laquelle il appartient. Des fichiers comportent des données chiffrées associées à un indice, il nous a fallu les séparer afin de pouvoir conserver des valeurs exploitables. Toutes les colonnes ont aussi été typées afin que les données soient le plus cohérentes possible avec le format qu'elles étaient censées avoir.

Chaque colonne a bien entendu été renommée afin de garantir une unité dans les champs que nous explorerons par la suite au sein du projet. Chaque donnée non complète a été écartée, afin d'utiliser un jeu de données final complet.


```
-- Création des tables de référence
CREATE TABLE IF NOT EXISTS pays (
    id_pays SERIAL PRIMARY KEY,
    nom_pays VARCHAR(100) NOT NULL,
    region VARCHAR(100),
    sous_region VARCHAR(100)
);

CREATE TABLE IF NOT EXISTS unite (
    id_unite SERIAL PRIMARY KEY,
    nom_unite VARCHAR(50) NOT NULL
);

CREATE TABLE IF NOT EXISTS type_statistique (
    id_type_statistique SERIAL PRIMARY KEY,
    nom_type_statistique VARCHAR(100) NOT NULL
);

CREATE TABLE IF NOT EXISTS type_traitement (
    id_type_traitement SERIAL PRIMARY KEY,
    nom_type_traitement VARCHAR(100) NOT NULL
);

-- Création des tables principales
CREATE TABLE IF NOT EXISTS population_hiv (
    id SERIAL PRIMARY KEY,
    id_pays INTEGER REFERENCES pays(id_pays),
    annee INTEGER,
    valeur DECIMAL(10,2),
    id_unite INTEGER REFERENCES unite(id_unite),
    UNIQUE(id_pays, annee)
```

Dans le fichier de population atteinte par le VIH, nous avons commencé par extraire les valeurs médianes, minimales et maximales en suivant un ensemble de règles de transformation, comme on le voit sur l'image ci-dessus. Une fois découpées, ces données ont été réparties selon trois colonnes, les valeurs nulles ou vides ont été supprimées, et enfin les données ont été converties en entier avant l'organisation finale des colonnes.

API : Obtention et manipulation des données

I. Mise en place de l'API

Nous avons choisi Python comme environnement afin de déployer notre API, étant l'un des langages de programmation les plus populaires et bénéficiant de nombreuses ressources en ligne grâce à sa communauté très développée. Le fait que ce langage soit flexible et adaptatif, doté d'une syntaxe claire et facile à mettre en œuvre, ainsi que le langage phare dans le domaine de l'IA et de l'analyse de données ont été des facteurs décisifs dans notre choix. (voir annexe)

Dans un premier temps nous nous sommes appuyés sur nos schémas de base de données afin de concevoir l'architecture de notre API. Nous avons utilisé des dépendances simples d'utilisation, afin de garantir une maintenabilité relativement aisée du code (FastAPI, SQLAlchemy, Pydantic, PyMySQL, python-dotenv) et capable de corréler les informations que nous avons stocké en BDD (MySQL).

```
schemas.py > ...
1  from pydantic import BaseModel
2  from typing import Optional
3
4
5  #schemas des pays
6  class PaysBase(BaseModel):
7      nom: str
8      region: Optional[str] = None
9
10 class PaysCreate(PaysBase):
11     pass
12
13 class Pays(PaysBase):
14     id_pays: int
15
16     class Config:
17         orm_mode = True
18
19 #schémas des unités
20 class UniteBase(BaseModel):
21     libelle: str
22
23 class UniteCreate(UniteBase):
24     pass
25
26 class Unite(UniteBase):
27     id_unite: int
28
29     class Config:
30         orm_mode = True
31
```

Nous avons défini les modèles (SQLAlchemy) pour représenter les tables de la BDD en configurant les clés primaires et auto-incrémentées afin d'éviter les erreurs de duplication. Nous y avons associé des schémas Pydantic afin de valider les données entrantes et sortantes pour s'assurer de la qualité des informations retournées. Enfin, nous avons mis en place un ensemble de chemins d'accès (endpoints) qui respectent le fonctionnement d'un CRUD (Create, Read, Update, Delete) afin que les utilisateurs puissent se servir de l'api pour obtenir et/ou manipuler les informations que contiennent chaque table.

II. Documentation de l'API

L'API dispose d'une documentation interactive générée par FastAPI, que l'on peut consulter via Swagger ou ReDoc. Vous trouverez plus de détails sur son fonctionnement dans le document joint appelé « Documentation API » ainsi que des exemples visuels détaillés fournis en annexe.

default



POST	/pays/	Create Pays	▼
GET	/pays/	Read Pays	▼
GET	/pays/{id_pays}	Read Pays By Id	▼
PUT	/pays/{id_pays}	Update Pays	▼
DELETE	/pays/{id_pays}	Delete Pays	▼
POST	/unite/	Create Unite	▼
GET	/unite/	Read Unite	▼
GET	/unite/{id_unite}	Read Unite By Id	▼
PUT	/unite/{id_unite}	Update Unite	▼
DELETE	/unite/{id_unite}	Delete Unite	▼
POST	/type_statistique/	Create Type Statistique	▼
GET	/type_statistique/	Read Type Statistiques	▼

PowerBI : Solution de visualisation

I. Benchmark

Afin de visualiser les informations retournées par nos ETL, nous avons soumis quatre logiciels à notre banc d'essai. Nous avons comparé Power BI, Tableau, Grafana et Matplotlib selon les

critères de performance, leur facilité d'utilisation, l'interactivité du produit, sa compatibilité avec les différents formats de fichiers, et enfin leur coût.

Nous avons orienté notre choix sur PowerBi qui répond totalement à nos attentes. Le fait qu'il soit compatible avec les données que nous utilisons associé à un coût d'utilisation faible et modulable selon les besoins et les usages, ainsi qu'une facilité d'utilisation grâce à son interface simple et pratique a achevé de nous convaincre.

II. Explications des visualisations

Conclusion

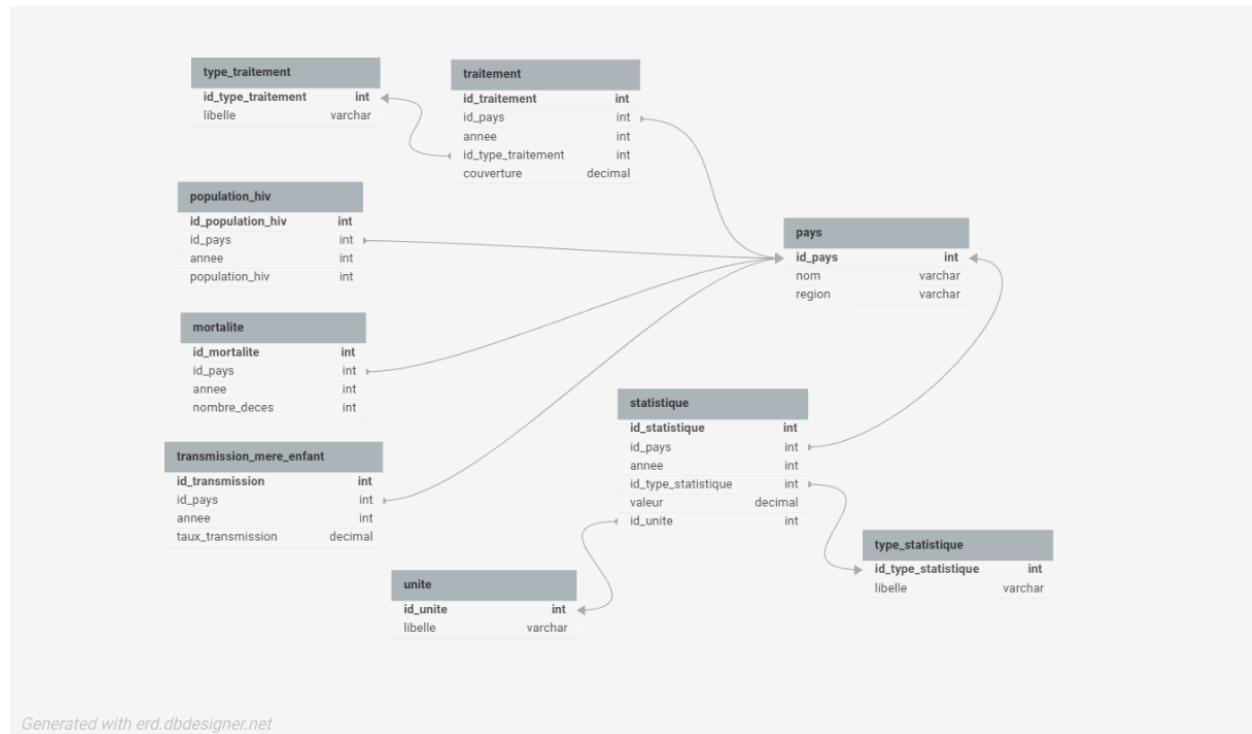
En résumé, ce projet permet d'ouvrir la voie vers une solution basée sur l'intelligence artificielle. Les différentes étapes suivies, allant de la modélisation de la base de données à la création de visualisations interactives avec Power BI, ont démontré l'importance d'une architecture de données bien pensée, d'un processus ETL rigoureux et d'une API performante. Grâce à cette démarche méthodologique, nous avons pu extraire des informations pertinentes et stratégiques, facilitant ainsi la compréhension et l'analyse de l'épidémie du VIH.

Il sera bon d'envisager l'utilisation de set de données complémentaires, afin de renforcer les résultats de notre travail et d'offrir une meilleure vue d'ensemble du Virus, en ajoutant les données récoltées au cours du temps par exemple. Afin d'enrichir les perspectives potentielles qu'offre la visualisation de ces données.

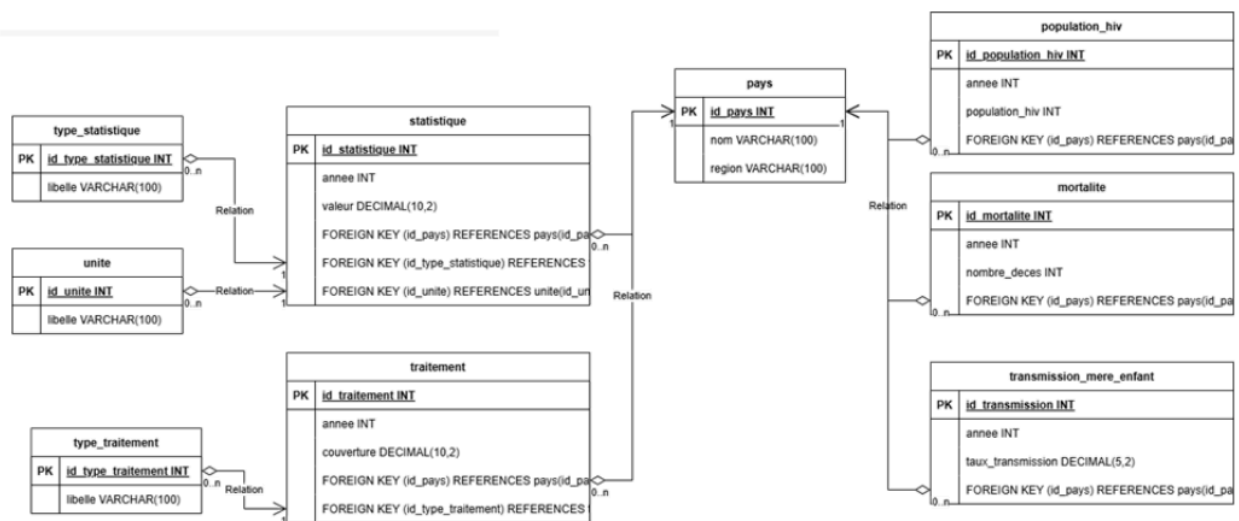
Les résultats obtenus montrent que l'exploitation des données à travers des outils modernes et adaptés peut non seulement améliorer la qualité des analyses, mais également apporter des insights précieux pour la lutte contre des épidémies telles que le VIH. En somme, ce projet met en lumière le potentiel des solutions basées sur l'intelligence artificielle pour répondre aux défis sanitaires actuels et futurs.

Annexe

Modèle Physique de données



Modèle Conceptuel de Données



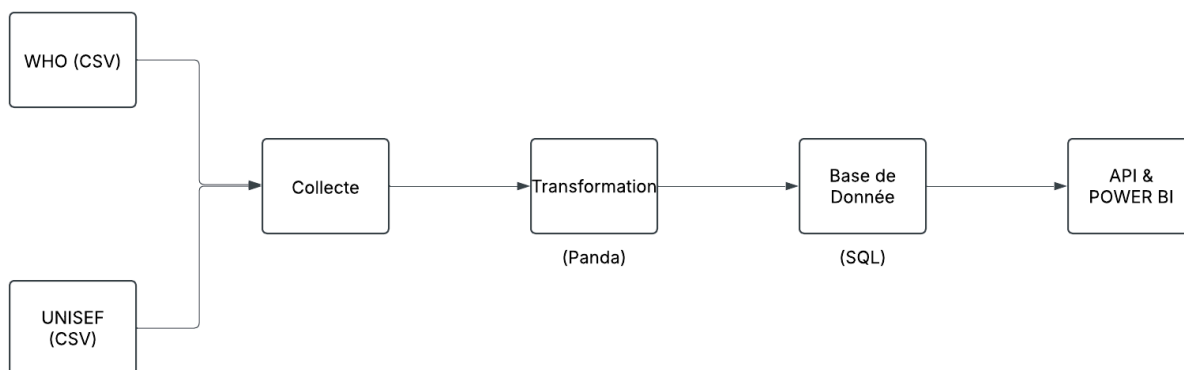
Benchmark ETL

Benchmark des solutions de collecte et transformation des données (ETL)

Critères	Pandas (choisi)	Talend	Apache Hop	Airflow
Performance	Excellente pour les données de taille moyenne, optimisable avec Dask	Bonne pour de gros volumes de données	Adapté aux projets ETL modulaires	Excellente pour orchestrer des workflows
Facilité d'utilisation	Simple pour les développeurs Python	Interface no-code, prise en main rapide	Interface graphique et scriptable	Plus complexe à configurer
Scalabilité	Nécessite Dask/Spark pour très gros volumes	Adapté aux grands volumes	Bonne scalabilité	Idéal pour le big data
Compatibilité	JSON, CSV, SQL, API	Nombreux connecteurs (BD, API, fichiers)	Connecteurs variés	Forte intégration avec le cloud
Coût	Gratuit et Open Source	Version gratuite + payante	Open Source	Open Source

Illustrations du Processus de Collecte et Traitement des Données

Pipeline de Collecte des Données



ETL | Nettoyage et Transformation des Données



ETL | Step-by-Step Avec Pandas



Capture du code ETL


```

# ----- ● EXTRACTION (Extract) -----
Tabnine | Edit | Test | Explain | Document
def extract_data():
    """
    EXTRACTION : Lecture des fichiers sources
    Returns:
        list: Liste des DataFrames extraits ou None en cas d'erreur
    """
    try:
        logging.info("📄 Début de l'extraction...")

        files = [
            "../SourceData/art_coverage_by_country_clean.csv",
            "../SourceData/art_pediatric_coverage_by_country_clean.csv",
            "../SourceData/no_of_cases_adults_15_to_49_by_country_clean.csv",
            "../SourceData/no_of_deaths_by_country_clean.csv",
            "../SourceData/no_of_people_living_with_hiv_by_country_clean.csv",
            "../SourceData/prevention_of_mother_to_child_transmission_by_country_clean.csv"
        ]

        dataframes = []
        for file_path in files:
            path = Path(file_path)
            if not path.exists():
                raise FileNotFoundError(f"Fichier introuvable : {file_path}")

            df = pd.read_csv(file_path)
            if df.empty:
                logging.warning(f"⚠ Le fichier {file_path} est vide")
                continue

```

```

# Vérification des colonnes nécessaires
country_col = next((col for col in df.columns if "Country" in col), None)
region_col = next((col for col in df.columns if "WHO Region" in col), None)

.....

if country_col:
    df.rename(columns={country_col: "pays"}, inplace=True)
    if region_col:
        df.rename(columns={region_col: "region_who"}, inplace=True)
        df = df[["pays", "region_who"]].drop_duplicates()
    else:
        df = df[["pays"]].drop_duplicates()
    dataframes.append(df)
else:
    logging.warning(f"⚠️ Aucune colonne 'Country' trouvée dans {file_path}")
    continue

.....

if not dataframes:
    raise ValueError("Aucune donnée valide extraite des fichiers sources")

.....

logging.info("✅ Extraction réussie")
return dataframes

.....

except FileNotFoundError as e:
    logging.error(f"❌ Erreur d'accès fichier : {str(e)}")
    return None
except ValueError as e:
    logging.error(f"❌ Erreur de données : {str(e)}")
    return None
except Exception as e:
    logging.error(f"❌ Erreur inattendue lors de l'extraction : {str(e)}")

```

```

# ----- ● TRANSFORMATION (Transform) -----
Tabnine | Edit | Test | Explain | Document
def transform_data(dataframes):
    """
    TRANSFORMATION : Nettoyage et structuration des données
    Args:
        dataframes (list): Liste des DataFrames à transformer
    Returns:
        DataFrame: Données transformées ou None en cas d'erreur
    """
    try:
        logging.info("🔔 Début de la transformation...")

        # 1. Concaténation des DataFrames
        try:
            pays_df = pd.concat(dataframes, ignore_index=True)
            if pays_df.empty:
                raise ValueError("Aucune donnée après concaténation")
        except Exception as e:
            raise ValueError(f"Erreur lors de la concaténation : {str(e)}")

        # 2. Nettoyage des données
        try:
            # Standardisation des noms de pays
            pays_df["pays"] = pays_df["pays"].str.strip().str.lower().str.replace(r"\s+", " ", regex=True)

            # Nettoyage des régions si présentes
            if "region_who" in pays_df.columns:
                pays_df["region_who"] = pays_df["region_who"].str.strip().str.lower()

```

```
        # Suppression des doublons
        pays_df.drop_duplicates(subset=["pays"], inplace=True)

        .....

        if pays_df.empty:
            raise ValueError("Aucune donnée valide après nettoyage")

        .....

        except Exception as e:
            raise ValueError(f"Erreur lors du nettoyage : {str(e)}")

        .....

        # 3. Ajout des identifiants
        pays_df.insert(0, "id_pays", range(1, len(pays_df) + 1))

        .....

        logging.info("✅ Transformation réussie")
        return pays_df

        .....

    except ValueError as e:
        logging.error(f"❌ Erreur de transformation : {str(e)}")
        return None

    except Exception as e:
        logging.error(f"❌ Erreur inattendue lors de la transformation : {str(e)}")
        return None
```

Capture du code API – classe python de la BDD

```

models.py > TransmissionMereEnfant
1  from sqlalchemy import Column, Integer, Sequence, String, DECIMAL, ForeignKey
2  from sqlalchemy.ext.declarative import declarative_base
3  from sqlalchemy.orm import relationship
4
5  Base = declarative_base()
6
7  class Pays(Base):
8      __tablename__ = "pays"
9      id_pays = Column(Integer, Sequence('pays_id_pays_seq'), primary_key=True, index=True)
10     nom = Column(String(100))
11     region = Column(String(100))
12
13     class Unite(Base):
14         __tablename__ = "unite"
15         id_unite = Column(Integer, Sequence('unite_id_unite_seq'), primary_key=True, index=True)
16         libelle = Column(String(100))
17
18     class TypeStatistique(Base):
19         __tablename__ = "type_statistique"
20         id_type_statistique = Column(Integer, Sequence('type_statistique_id_type_statistique_seq'), primary_key=True, index=True)
21         libelle = Column(String(100))
22
23     class TypeTraitement(Base):
24         __tablename__ = "type_traitement"
25         id_type_traitement = Column(Integer, Sequence('type_traitement_id_type_traitement_seq'), primary_key=True, index=True)
26         libelle = Column(String(100))
27
28     class Traitement(Base):
29         __tablename__ = "traitement"
30         id_traitement = Column(Integer, Sequence('traitement_id_traitement_seq'), primary_key=True, index=True)
31         id_pays = Column(Integer, ForeignKey("pays.id_pays"))
32         annee = Column(Integer)
33         id_type_traitement = Column(Integer, ForeignKey("type_traitement.id_type_traitement"))
34         couverture = Column(DECIMAL(10, 2))
35

```

Capture code API – connexion BDD suivi des endpoints concernant la table pays

```
database.py > ...
1  from sqlalchemy import create_engine
2  from sqlalchemy.ext.asyncio import AsyncSession, create_async_engine
3  from sqlalchemy.ext.declarative import declarative_base
4  from sqlalchemy.orm import sessionmaker
5  from dotenv import load_dotenv
6  import os
7
8  load_dotenv() # Charger les variables d'environnement depuis le fichier .env
9
10 SQLALCHEMY_DATABASE_URL = os.getenv("DATABASE_URL")
11
12 engine = create_engine(SQLALCHEMY_DATABASE_URL, echo=True)
13 SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
14 Base = declarative_base()
15
16 def get_db():
17     db = SessionLocal()
18     try:
19         yield db
20     finally:
21         db.close()
22
```

```

# Create (Ajouter un nouveau pays)
@app.post("/pays/", response_model=Pays)
def create_pays(pays: PaysCreate, db: Session = Depends(get_db)):
    try:
        db_pays = PaysModel(**pays.dict())
        db.add(db_pays)
        db.commit()
        db.refresh(db_pays)
        return db_pays
    except Exception as e:
        db.rollback() # Annuler la transaction en cas d'erreur
        raise HTTPException(status_code=500, detail=str(e)) # Lever une exception avec les détails de l'erreur

# Read (voir les pays)
@app.get("/pays/", response_model=List[Pays])
def read_pays(skip: int = 0, limit: int = 10, db: Session = Depends(get_db)):
    return db.query(PaysModel).offset(skip).limit(limit).all()

# Read (Voir un pays en fonction de son id)
@app.get("/pays/{id_pays}", response_model=Pays)
def read_pays_by_id(id_pays: int, db: Session = Depends(get_db)):
    db_pays = db.query(PaysModel).filter(PaysModel.id_pays == id_pays).first()
    if db_pays is None:
        raise HTTPException(status_code=404, detail="Pays not found")
    return db_pays

```

```

# Update ( modifier les informations d'un pays en fonction de son id)
@app.put("/pays/{id_pays}", response_model=Pays)
def update_pays(id_pays: int, pays: PaysCreate, db: Session = Depends(get_db)):
    db_pays = db.query(PaysModel).filter(PaysModel.id_pays == id_pays).first()
    if db_pays is None:
        raise HTTPException(status_code=404, detail="Pays not found")
    db_pays.nom = pays.nom
    db_pays.region = pays.region
    db.commit()
    db.refresh(db_pays)
    return db_pays

# Delete (supprimer les informations d'un pays)
@app.delete("/pays/{id_pays}", response_model=Pays)
def delete_pays(id_pays: int, db: Session = Depends(get_db)):
    db_pays = db.query(PaysModel).filter(PaysModel.id_pays == id_pays).first()
    if db_pays is None:
        raise HTTPException(status_code=404, detail="Pays not found")
    db.delete(db_pays)
    db.commit()
    return {"detail": "Pays deleted"}

```

Image Documentation swagger et ReDoc

POST **/pays/** Create Pays ^

Parameters Try it out

No parameters

Request body required application/json ▼

Example Value | **Schema**

```
{
  "nom": "string",
  "region": "string"
}
```

Responses

Code	Description	Links
200	Successful Response	No links
	Media type application/json ▼ Controls Accept header. Example Value Schema <pre>{ "nom": "string", "region": "string", "id_pays": 0 }</pre>	
422	Validation Error	No links

422 Validation Error No links

Media type
application/json

Example Value | Schema

```
{  "detail": [    {      "loc": [        "string",        0      ],      "msg": "string",      "type": "string"    }  ]}
```

Search...

- POST** Create Pays
- GET** Read Pays
- GET** Read Pays By Id
- PUT** Update Pays
- DEL** Delete Pays
- POST** Create Unite
- GET** Read Unite
- GET** Read Unite By Id
- PUT** Update Unite
- DEL** Delete Unite
- POST** Create Type Statistique
- GET** Read Type Statistiques
- GET** Read Type Statistique By Id
- PUT** Update Type Statistique
- DEL** Delete Type Statistique
- POST** Create Type Traitement

[API docs by Redocly](#)

FastAPI (0.1.0)

Download OpenAPI specification: [Download](#)

Create Pays

REQUEST BODY SCHEMA: application/json

nom required string (Nom)
region > Region (string) or Region (null) (Region)

Responses

> **200** Successful Response

> **422** Validation Error

POST /pays/

Request samples

Payload

Content type
application/json

Copy

```
{  "nom": "string",  "region": "string"}
```

Response samples

200 **422**

Content type
application/json

Copy

```
{  "nom": "string",
```

Benchmark Visualisation Data

Benchmark des solutions de visualisation des données

Critères	Power BI (choisi)	Tableau	Grafana	Matplotlib / Seaborn
Performance	Excellente pour des grands jeux de données	Très puissant, bien optimisé	Bon pour données en temps réel	Dépend des capacités Python
Facilité d'utilisation	Interface intuitive avec glisser-déposer	Interface puissante mais plus complexe	Simple pour monitoring	Requiert du code Python
Interactivité	Tableaux de bord interactifs avec filtres et drill-down	Très interactif et esthétique	Axé monitoring et alertes	Statique (peut être enrichi avec Dash)
Comptabilité	CSV, SQL, Excel, API	Connecte à de nombreuses sources	Bases de données, JSON	Intégré à Python, supporte CSV, JSON, SQL
Coût	Gratuit (avec version Pro payante)	Payant (cher pour entreprises)	Gratuit	Gratuit